A D PATEL INSTITUTE OF TECHNOLOGY

(The Charutar Vidya Mandal University)

TO-DO LIST

A Python Programming Mini-Project Report

A project report submitted in partial fulfilment of the requirements for the subject

PYTHON PROGRAMMING

Bachelor of Technology in Information Technology

Semester V

Submitted by

Ayaz Vhora

12402080601011

Faculty Guide

Prof. Nayan M.Mali

Academic Year 2026–2027 (Odd)

Department of Information Technology

A D PATEL INSTITUTE OF TECHNOLOGY

(The Charutar Vidya Mandal University)

New Vallabh Vidyanagar, Anand, Gujarat, India

A D PATEL INSTITUTE OF TECHNOLOGY

(The Charutar Vidya Mandal University)

CERTIFICATE

This is to certify that the project report entitled **"To-Do List"** has been prepared by Ayaz Vhora (12402080601011), a student of the Department of Information Technology, A D PATEL INSTITUTE OF TECHNOLOGY, in partial fulfilment of the academic requirements for the subject Python Programming.

The report presents a beginner-level Python program for maintaining a list of tasks. It explains the purpose, requirements, program design, algorithms, implementation, sample interactions, testing approach, limitations and possible future improvements.

The project is intended for academic learning and demonstrates fundamental Python concepts such as lists, functions, loops, conditional statements, user input and output. The final implementation and test results should be checked against the source code submitted by the student.

Faculty Guide: Prof. Nayan M.Mali

Head of Department: Priyang Bhatt

Name:Ayaz Vhora
En No:12402080601011

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my faculty guide, Prof. Nayan M.Mali, for her guidance and encouragement during the preparation of this Python Programming mini-project report.

I am thankful to the Department of Information Technology and A D PATEL INSTITUTE OF TECHNOLOGY for providing the opportunity to apply programming concepts to a practical problem. This project helped me understand how Python lists can store tasks and how functions can organise operations such as adding, viewing, completing and deleting tasks.

I also thank my classmates, friends and family for their support and motivation. Preparing this project strengthened my understanding of problem-solving, program flow, input validation and testing. I have tried to keep the implementation simple and suitable for beginners.

Ayaz Vhora

12402080601011

ABSTRACT

The To-Do List is a beginner-level Python console project designed to help users organise everyday tasks. It provides a simple menu through which a user can add a task, view all tasks, mark a task as completed, remove a task and exit the program. The project demonstrates how a small program can make task tracking clearer than keeping an unstructured handwritten list.

The program uses a Python list to store task records. Each task can be represented by a dictionary containing a task description and a completion status. Functions separate the main operations, a loop repeatedly displays the menu, and conditional statements select the required action. The input() and print() functions provide a straightforward console interface.

This report describes the project objectives, scope, software requirements, data representation, workflow, algorithms, sample code, example input and output, testing cases, complexity, limitations and future improvements. The basic version stores data in memory, so tasks are not retained after the program closes unless file or database storage is added. It is an educational prototype rather than a full task-management application.

Keywords: Python, To-Do List, task management, list, dictionary, functions, loops, conditional statements.

Name:Ayaz Vhora
En No:12402080601011

TABLE OF CONTENTS

Certificate 2

Acknowledgement 3

Abstract 4

Chapter 1 – Introduction and Objectives 6

1.1 Background and Introduction 6

1.2 Problem Statement 6

1.3 Objectives and Scope 6

1.4 Key Features and Benefits 6

Chapter 2 – Technology and Requirements 7

2.1 Python and Data Structures 7

2.2 Functional Requirements 7

2.3 Hardware and Software Requirements 7

2.4 Development Methodology 7

Chapter 3 – System Design and Algorithms 8

3.1 System Workflow 8

3.2 Task Representation 8

3.3 Algorithms and Pseudocode 8

3.4 Complexity Analysis 8

Chapter 4 – Implementation and Sample Output 9

4.1 Program Code 9

4.2 Sample Interaction 9

Chapter 5 – Testing, Limitations and Conclusion 10

5.1 Testing 10

5.2 Limitations and Future Scope 10

5.3 Conclusion 10

References 10

Name:Ayaz Vhora
En No:12402080601011

CHAPTER 1 – INTRODUCTION AND OBJECTIVES

1.1 Background and Introduction

People often need to remember assignments, personal errands, study activities and other daily responsibilities. When tasks are not recorded in an organised way, important work may be forgotten or completed late. A To-Do List is a simple method of writing down tasks and tracking whether each task is still pending or has been completed.

1.2 Problem Statement

A handwritten list can be difficult to update when tasks are added, completed or cancelled. The purpose of this project is to demonstrate how a small Python program can organise tasks through a menu-driven interface. The program provides basic operations without requiring a graphical interface or external libraries.

1.3 Project Objectives

The main objective is to develop a simple and understandable Python program for managing tasks. Specific objectives are to: (1) add new tasks; (2) display saved tasks; (3) mark a task as completed; (4) delete a task when it is no longer required; (5) use lists, dictionaries, functions, loops and conditions; and (6) practise testing and debugging.

1.4 Scope

The project covers task entry, task listing, completion status, task deletion and menu-based navigation during one program session. It is suitable for beginner-level classroom demonstration and can be extended later with file storage, due dates or a graphical interface.

1.5 Key Features and Benefits

The menu makes operations easy to select. A numbered task list helps users identify a task for completion or deletion. Separating operations into functions improves readability and maintenance. The project also provides practical experience in translating a small real-world problem into a sequence of algorithmic steps.

CHAPTER 2 – TECHNOLOGY AND REQUIREMENTS

2.1 Python Programming Language

Python is used because its syntax is readable and suitable for beginners. The basic version relies on built-in language features, so no third-party package is required. The program can be run from a terminal or from Visual Studio Code with a Python interpreter installed.

2.2 Lists and Dictionaries

A list named tasks stores all task records. Each record is a dictionary, for example {'title': 'Complete assignment', 'completed': False}. The title stores the task description, while the Boolean completion field records whether the task is pending or complete. List indexes are used to select a task from the displayed list.

2.3 Functions, Loops and Conditions

Functions divide the program into focused operations such as add_task(), view_tasks(), complete_task() and delete_task(). A while loop repeats the menu until the user chooses Exit. if/elif/else statements choose the operation and handle invalid menu selections. A for loop displays each task.

2.4 Functional Requirements

The system should display a menu, accept a task description, store valid tasks, show all tasks with their status, mark a selected task as completed, delete a selected task, provide a message when the task list is empty and exit when requested.

2.5 Non-Functional Requirements

Usability: menu options and messages should be clear. Reliability: valid inputs should produce the expected result. Maintainability: functions should have separate responsibilities. Portability: the program should run on systems with Python 3. Safety: invalid task numbers and empty descriptions should be handled without an unexpected crash.

2.6 Hardware and Software Requirements

Hardware: a basic desktop or laptop computer. Software: Windows, Linux or macOS; Python 3; and an editor such as Visual Studio Code. Internet access is not required to run the basic program.

2.7 Development Methodology

The project can be developed incrementally: define the task data structure, create the menu, implement adding and viewing, add completion and deletion, validate user input, and test each operation. Small steps make errors easier to locate and correct.

CHAPTER 3 – SYSTEM DESIGN AND ALGORITHMS

3.1 System Workflow

The program follows an input-process-output model. The user selects an option, enters a task or task number, the relevant function reads or updates the tasks list, and a message is displayed. The menu then appears again until Exit is selected.

3.2 High-Level Design

User → Console Menu → Selected Function → Tasks List → Console Output. The list is held in memory while the program runs. In the basic implementation, there is no permanent storage, user login or online synchronisation.

3.3 Algorithm: Add a Task

Step 1: Ask the user to enter a task description. Step 2: Remove unnecessary leading and trailing spaces. Step 3: If the description is empty, show an error message. Step 4: Otherwise, create a task record with completed set to False. Step 5: Append the record to the list and show a confirmation message.

3.4 Algorithm: View Tasks

Step 1: Check whether the tasks list is empty. Step 2: If it is empty, display a message. Step 3: Otherwise, visit each task in the list. Step 4: Print its number, description and completion status.

3.5 Algorithm: Complete or Delete a Task

Ask the user for the displayed task number. Convert it to a list index by subtracting one. Check that the number is within the valid range. For completion, update the task status to True. For deletion, remove the selected record and display a confirmation. **3.6 Pseudocode**

```
BEGIN tasks
empty list REPEAT
DISPLAY menu
INPUT choice
IF choice = Add THEN add task
ELSE IF choice = View THEN display tasks
ELSE IF choice = Complete THEN update selected task
ELSE IF choice = Delete THEN remove selected task
ELSE IF choice = Exit THEN stop
ELSE DISPLAY invalid choice
UNTIL choice = Exit
END
```

3.7 Complexity Analysis

Let n be the number of tasks. Appending a task to a Python list is amortised O(1). Displaying all tasks is O(n). Selecting a task by list index is O(1), while deleting an item from the middle of a list is O(n) because later elements may shift. Storage is O(n) because each task is kept in the list.

CHAPTER 4 – IMPLEMENTATION AND SAMPLE OUTPUT

4.1 Example Python Program

The following is a simple reference implementation. Save it as todo_list.py. It keeps tasks in memory and includes basic checks for empty lists and invalid task numbers.

```
tasks = []

def add_task():
    title = input("Enter task: ").strip()     if not
title:         print("Task cannot be empty.")
return     tasks.append({"title": title, "completed":
False})     print("Task added successfully.")

def view_tasks():     if not tasks:
print("No tasks available.")         return
for i, task in enumerate(tasks, start=1):
        status = "Completed" if task["completed"] else "Pending"
print(f"{i}. {task['title']} - {status}")

def choose_task():
try:
        number = int(input("Enter task number: "))
if 1 <= number <= len(tasks):
            return number - 1
except ValueError:
        pass     print("Invalid
task number.")     return None

def complete_task():
    index = choose_task()
if index is not None:
        tasks[index]["completed"] = True
print("Task marked as completed.")

def delete_task():
    index = choose_task()
if index is not None:
        removed = tasks.pop(index)
print("Deleted:", removed["title"])

while True:
    print("\n1.Add  2.View  3.Complete  4.Delete  5.Exit")
choice = input("Choose option: ")       if choice == "1":
add_task()      elif choice == "2":           view_tasks()
elif choice == "3":         view_tasks()         if tasks:
complete_task()     elif choice == "4":        view_tasks()
if  tasks:  delete_task()           elif  choice  ==  "5":
print("Goodbye!")       break    else:       print("Invalid
choice. Try again.")
```

4.2 How to Run

1. Open the project folder in Visual Studio Code. 2. Create a file named todo_list.py. 3. Paste the code and save it. 4. Open Terminal → New Terminal. 5. Run: python todo_list.py. If required on Windows, try: py todo_list.py.

4.3 Sample Interaction

```
Choose option: 1
Enter task: Complete Python assignment
Task added successfully.

Choose option: 1
Enter task: Revise for exam
Task added successfully.

Choose option: 2
1. Complete Python assignment - Pending
2. Revise for exam - Pending

Choose option: 3
Enter task number: 1 Task
marked as completed.

Choose option: 2
1. Complete Python assignment - Completed


2. Revise for exam - Pending
```

A D PATEL INSTITUTE OF TECHNOLOGY

(The Charutar Vidya Mandal University)

CHAPTER 5 – TESTING, LIMITATIONS AND CONCLUSION

5.1 Testing Approach

Testing checks whether each menu option behaves as expected. The following cases should be tried using the actual source code before submission.

5.2 Current Limitations

Tasks are stored only in memory and disappear when the program closes. The interface is text-based, and there is no login, reminder notification, due-date management, cloud synchronisation or shared task list. The basic version is intended for learning and should not be described as a complete commercial productivity application.

5.3 Future Improvements

Possible improvements include saving tasks to a text or JSON file, adding due dates and priorities, searching tasks by keyword, filtering pending and completed tasks, adding confirmation before deletion, and developing a graphical interface. These features should be introduced gradually and tested separately.

5.4 Conclusion

The To-Do List project demonstrates how Python can be used to solve a simple task-organisation problem. Lists and dictionaries store task information, functions organise the operations, loops repeat the menu, and conditional statements control program flow. The project gives beginner-level practice in coding, algorithm design, input validation and testing.

5.5 Learning Outcomes

Through this project, a student can practise writing functions, using list indexes, updating dictionary values, handling invalid numeric input, tracing program execution and analysing basic time and space complexity. The project also shows how a simple prototype can be extended with persistent storage and additional features.

Test case	Input / action	Expected result
Add a valid task	Enter “Study Python”	Task is added
Add an empty task	Press Enter without text	Validation message appears
View empty list	Choose View before adding tasks	“No tasks available” appears
Complete a task	Choose a valid task number	Status becomes Completed
Invalid task number	Enter 0 or a number too large	Invalid task message appears
Delete a task	Choose a valid task number	Selected task is removed
Invalid menu choice	Enter an unsupported option	Menu displays an error message
Exit	Choose option 5	Program terminates

Name:Ayaz Vhora
En No:12402080601011

A D PATEL INSTITUTE OF TECHNOLOGY

(The Charutar Vidya Mandal University)

References

1. Python Software Foundation, Python 3 Documentation: https://docs.python.org/3/
2. Python Software Foundation, Built-in Types — Lists and Dictionaries: https://docs.python.org/3/library/stdtypes.html
3. Python Software Foundation, Errors and Exceptions: https://docs.python.org/3/tutorial/errors.html
4. Course material: Python Programming — functions, lists, loops, conditions and basic algorithm design.

Name:Ayaz Vhora
En No:12402080601011